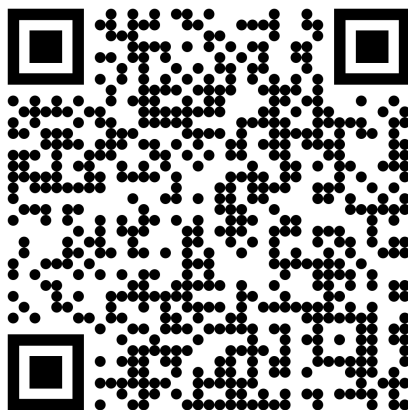


COMPUTER VISION PROJECT

CNN classifier

Davide Zorretto

January 2025



Github repository:

<https://github.com/Davidezorz/Computer-Vision-2025-CNN-classifier>

Contents

1	Introduction	3
2	Dataset Description	4
3	Dataset Preprocessing	4
4	Parser	6
5	CNN	7
6	Training Procedure	7
7	Main Solver	8
7.1	Outputs	9
8	Point 1	9
8.1	Network Architecture	10
8.2	Results Analysis	10
9	Point 2	12
9.1	Point 2a	12
9.1.1	Network Architecture	12
9.1.2	Data Pipeline	12
9.1.3	Results Analysis	12
9.2	Point 2b	14
9.2.1	Network Architecture	14
9.2.2	Data Pipeline	14
9.2.3	Results Analysis	14
9.3	Point 2c	16
9.3.1	Network Architecture	16
9.3.2	Data Pipeline	17
9.3.3	Results Analysis	17
9.4	Point 2d	18
9.4.1	Network Architecture	18
9.4.2	Data Pipeline	19
9.4.3	Results Analysis	19
9.5	Point 2e	21
9.5.1	Network Architecture	21
9.5.2	Data Pipeline	21
9.5.3	Results Analysis	21
9.6	Point 2f	23
9.6.1	Network Architecture	23
9.6.2	Data Pipeline	23

9.6.3 Results Analysis	23
10 Point 3	24
10.1 Point 3a	24
10.1.1 Network Architecture	25
10.1.2 Data Pipeline	25
10.1.3 Results Analysis	25
10.2 Point 3b	27
10.2.1 Network Architecture	27
10.2.2 Multiclass SVM	27
10.2.3 Data Pipeline	27
10.2.4 Results Analysis	27
11 Point 4	28
11.0.1 Network Architecture	28
11.0.2 Data Pipeline	28
11.0.3 Results Analysis	29
12 Point 5	30
12.1 Network Architecture	30
12.2 Data Pipeline	31
12.3 Results Analysis	31
13 Point 6	33
13.1 Data Pipeline	33
13.2 Results Analysis	33
14 Point 7	33
14.1 Error Correcting Output Code (ECOC)	34
14.2 Data Pipeline	35
14.3 Results Analysis	35
15 Vision Transformer	36
15.1 Network Architecture	36
15.2 Results Analysis	37
16 Conclusion	38

1 Introduction

This report shows the implementation of an image classifiers trained on the provided dataset from [Lazebnik et al., 2006].

In particular, this project focused on convolutional neural networks (CNNs) as classifier or as feature extractors for then applying a multiclass SVM to get the predictions.

Each different point of this report will involve a change in the

- **model structure:** add convolution or change their properties, adding batch normalization and dropout;
- **data augmentation:** add random rotation, random cropping and random flip to the training images.

As code assistant, Gemini 3 was used in some cases to write better code or improving understanding, write better comments/improving english.

2 Dataset Description



Figure 1: 9 training image, true labels vs predicted by the point1 CNN

The dataset contains 4485 images with an heterogeneous resolution divided in 15 categories: *office, kitchen, living room, bedroom, store, industrial, tall building, inside city, street, highway, coast, open country, mountain, forest, suburb*.

It is already divided in training set and test set, having:

- **training set** with 1500 images, exactly 100 each category.
- **test set** with 2985 images, divided as

3 Dataset Preprocessing

The dataset preprocessing is done in file:

#	Class Name	Count	#	Class Name	Count
1	Bedroom	116	9	Mountain	274
2	Coast	260	10	Office	115
3	Forest	228	11	OpenCountry	310
4	Highway	160	12	Store	215
5	Industrial	211	13	Street	192
6	InsideCity	208	14	Suburb	141
7	Kitchen	110	15	TallBuilding	256
8	LivingRoom	189			

Table 1: Indexed Class Distribution

`dataset/dataloader.py`

In the class `DatasetManager`. When it is instantiated, it expect as input:

`folder_path (str)`: The main folder containing 'train' and 'test' folders.
`val_split (float)`: The percentage of training data to use for validation.
`pipe_base (list)`: Transforms applied for ALL images.
`pipe_train (list)`: Transforms ONLY for training.
`pipe_test (list)`: Transforms ONLY for validation/test.
`pipe_norm (list)`: Normalization values, applied at the very end.

All the pipelines lists are optional.

When calling the `get` method, which take the batch size as input, the function returns:

`data_loaders (list)`: it contains
 – `train_loader`: torch DataLoader containing training images divided in batches.
 – `val_loader`: torch DataLoader containing validation images divided in batches.
 – `test_loader`: torch DataLoader containing test images divided in batches.
`classes`: The list of class names found in the train folder.

The `get` method divide the dataset into training and test sets using a **stratified approach**, which guarantees that each class respects the specified validation proportion. The transformation for the **training, validation and test set** follows different procedures:

- **training set:**

1. It applies the base transformations (like resizing) if present

2. It applies the *training* specific transformations, (data augmentation, like random flips) to increase data variety
 3. It converts the images to Tensors
 4. It normalizes the images
- **validation set:** It applies the base transformations and optional test transformations, but it *skips* the random training augmentations:
 1. It applies the base transformations if any
 2. It applies the *test* specific transformations, if any are defined
 3. It converts the images to Tensors
 4. It normalizes the images
 - **test set:** It follows the exact same procedure as the validation set. It prepares the images without random augmentations to provide a fair final evaluation.

4 Parser

To make convolutional neural networks easier to write, a `convParser` class has been made. It can be found in:

`parsing/convParser.py`

The `str2dict` method takes as input a string, which describes the network configuration. It should be something in this form:

```
[module name] [arg]:[ values] [arg]:[ values] ...
[module name] [arg]:[ values] [arg]:[ values] ...
...
```

For example, the first network used for solving point 1 is written as

```
conv2d      channels: ->8    kernel_size: (3, 3)  stride: (1, 1)
relu
maxpool2d                    kernel_size: (2, 2)  stride: (2, 2)

conv2d      channels: ->16   kernel_size: (3, 3)  stride: (1, 1)
relu
maxpool2d                    kernel_size: (2, 2)  stride: (2, 2)

conv2d      channels: ->32   kernel_size: (3, 3)  stride: (1, 1)
relu

flatten
linear      dims:           ->15
```

At the end, it returns a list which contains a **list of dictionaries**. Each dictionary has the following keys:

Key	Description
type	The string identifier of the module (e.g., 'conv2d').
category	The internal category used for parsing logic (convolution, linear, function, or skip).
class	The reference to the actual PyTorch class object (e.g., <code>nn.Conv2d</code>).
args	A dictionary containing the parsed arguments (like kernel size or stride) used to initialize the class.

Table 2: Structure of the dictionary items returned by the parser.

5 CNN

The convolutional neural network class can be found in `models/generalCNN.py`

This class acts as a **dynamic Convolutional Neural Network** builder. It constructs a sequential PyTorch model directly from the list of configuration dictionaries, which are the output of the `parser.str2dict` method described previously.

A key feature is that it manages the transition of tensor shapes, from spatial dimensions to flattened vectors, automatically if a flatten module is written in the config. This **removes the need for manual dimension calculations** between blocks. The output dimensions are computed according to the formulas provided in the official PyTorch documentation¹.

Additionally, the class handles weight initialization automatically if no initialization is provided, applying by default *Kaiming Uniform* initialization to both convolutional and linear layers to ensure better convergence during training.

6 Training Procedure

The training file is the main folder, in the file `train.py`

¹<https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

which contains a `Validation` class and a `train` function. It implements a standard deep learning loop enhanced with the following features:

- **Loss Function:** It uses `CrossEntropyLoss`, which is the standard loss for multi-class classification problems.
- **Optimizer:** It uses the `Adam` optimizer by default.
- **Scheduler:** A `LinearLR` scheduler is applied. This gradually decreases the learning rate after every batch update, which helps the model settle into a better solution.
- **Automatic Mixed Precision (AMP):** The loop utilizes `torch.amp.GradScaler` and `autocast`. This allows the model to perform calculations using half-precision (float16) where safe. Since the documentation was a bit unclear, an LLM was used to write this procedure correctly. However, this feature will be **unused** in the final exercise solutions, as it did not provide a significant speed-up and makes training a little more unstable.
- **Validation and Early Stopping:** A specific `Validation` class handles the evaluation. It does not just calculate the loss; it actively monitors the training progress.
 1. It stores the weights of the **best model** found so far, the one with the lowest validation loss.
 2. It implements an **Early Stopping** mechanism: if the validation loss does not improve for a specific number of checks, defined by the `patience` parameter the training stops automatically to prevent overfitting.

At the end of the training, the function automatically reloads the best weights saved by the validation class, ensuring the returned model is the optimal one.

7 Main Solver

The main solver file is called `solve.py`. It can be found in the main folder and is able to solve the majority of the exercises. This script takes as input two arguments:

- **config_path:** The path to the configuration file, which contains all the necessary variable values. This file is organized into four main sections:
 - **save_path:** The folder where the results (plots, logs) will be stored.
 - **model:** Contains all values that describe the model, such as its structure, which will be processed by the parser, the initialization type, and its name (useful for saving files).
 - **training:** Contains all training hyperparameters, such as learning rate, batch size, learning rate decay, optimizer settings, and validation frequency.

- **data:** Defines the image transformations needed for the **base**, **train**, **test**, and **normalization** pipelines. These are passed to the `DatasetManager` class instance.

All the configuration files are located in the **configs folder** and have a `.yaml` extension, one for each exercise to enable maximal reproducibility.

- **train:** An optional boolean flag, true by default, that indicates whether training should be performed:
 - If **True**, the script runs the training loop.
 - If **False**, the script attempts to load pre-trained weights from the `.weights` folder and skips directly to evaluation.

Only `solve.py` handles correctly the **training** flag. The other solvers `point2f.py`, `point3a.py` and `pointSVM.py` will ignore this flag.

7.1 Outputs

Upon execution, the script automatically generates several files in the specified `save_path` to analyze the model’s performance:

1. **Training Plots:** If training is performed, it saves `model_name + _train_loss.png` and `model_name + _train_accuracy.png`, visualizing the metrics over epochs.
2. **Confusion Matrix:** It computes the confusion matrix on the test set and saves it as `model_name + _confusion_matrix.png`.
3. **Training and Test Images:** It stores `model_name + _test_examples.png` and `model_name + _train_examples.png`. These files contain nine sample images from the test and training sets respectively, showing a comparison of the true labels versus the predicted ones.
4. **Log File:** It saves a text file containing a summary of the run, including the configuration used, the total number of parameters, the training time, and the final test accuracy.

8 Point 1

This point can be solved using:

```
python solve.py --config_path='configs/point1.yaml'
```

8.1 Network Architecture

The network architecture is defined in `configs/point1.yaml`. It is the simple Convolutional Neural Network described in the assignment, with three convolutional blocks followed by a fully connected layer. All input images are resized to 64×64 pixels. The specific structure is:

- **Block 1:** Conv2d (8 filters, 3×3) $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2).
- **Block 2:** Conv2d (16 filters, 3×3) $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2).
- **Block 3:** Conv2d (32 filters, 3×3) $\rightarrow$ ReLU.
- **Classifier:** Flatten $\rightarrow$ Linear layer for the incoming dimension to 15, the output logits.

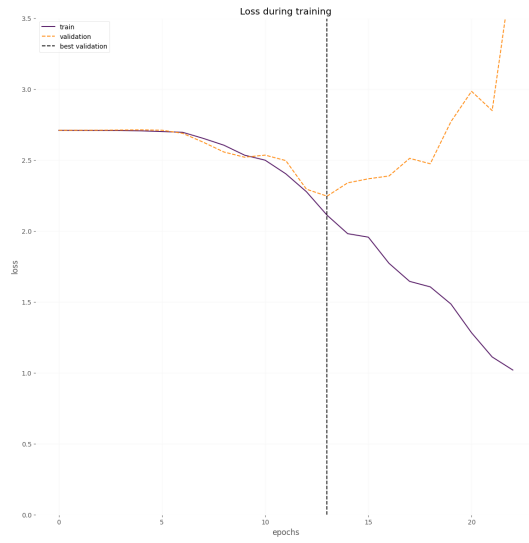
The total number of parameter is approximately 75 K .

The weights were initialized using a Normal distribution ($\sigma = 0.01$). The model was trained using **SGD** with a learning rate of 8×10^{-4} and momentum 0.9.

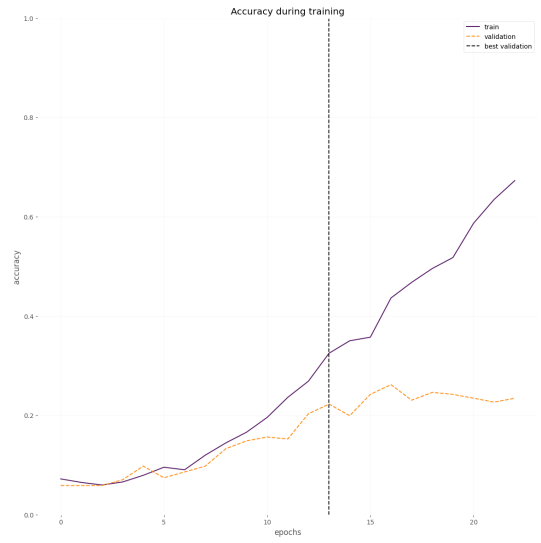
8.2 Results Analysis

The training results are shown in Figure 2. The model achieved a final accuracy on the test set of approximately **25.19%**.

- **Loss and Overfitting:** In the training plot we can see that the training loss, the purple line, and validation loss, the orange dashed line, diverges very quickly showing the model overfitting. The training loss continue decreasing while the validation loss after batch 13 starts increasing.
- **Best Model Selection:** The vertical dashed black line in the plots indicates the epoch where the **validation loss was at its minimum**. As already said, the training function automatically reload the best weights in the model, keeping a not overfitted model.
- **Confusion Matrix:** The Confusion Matrix (Figure 3) exhibits a slight diagonal, indicating that the model makes some correct predictions. However, classes like *Bedroom* and *Store* are frequently misclassified. It is evident that the model struggles to differentiate between visually similar classes.
- **Data Splitting Sensitivity:** The model reach $\approx 25\%$ accuracy differently from the $\approx 30\%$ expected. It was observed that the performance is highly sensitive to the numpy random seed used for splitting the training and validation sets. For example other splits (e.g., seed 18) resulted in better performance.



(a) Loss during training



(b) Accuracy during training

Figure 2: Training metrics for CNN_1. The vertical line marks the early stopping point.

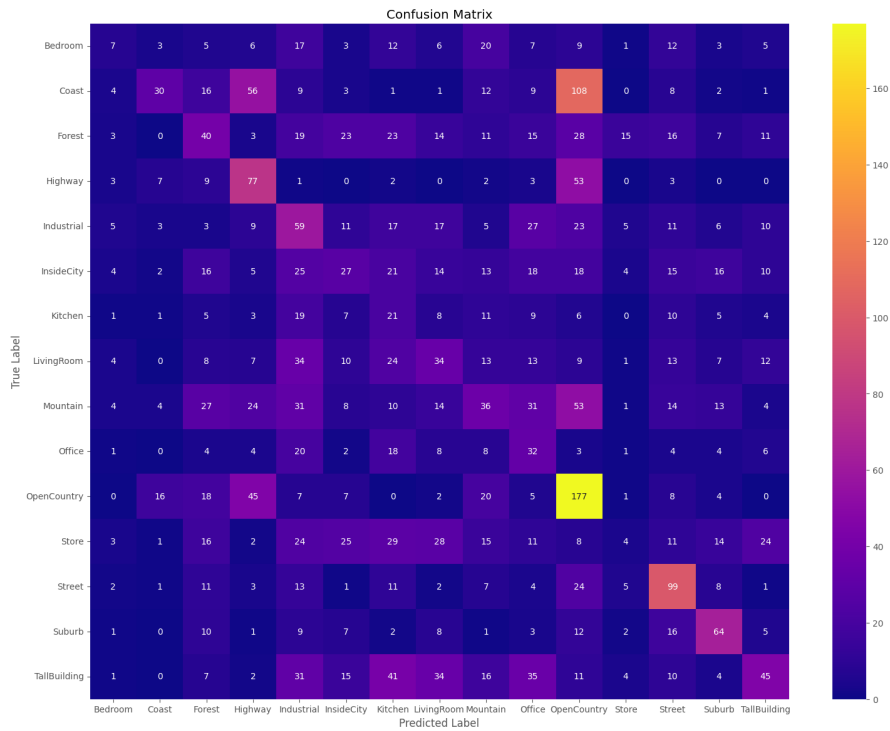


Figure 3: Confusion Matrix for CNN_1 on the test set.

9 Point 2

All exercises for point 2 can be solved using `solve.py`, except for the last point, which requires defining an ensemble CNN.

9.1 Point 2a

This point can be solved using:

```
python solve.py --config_path='configs/point2a.yaml'
```

9.1.1 Network Architecture

The network architecture is the same as in the previous point, with the same hyperparameters. It was, however, redefined in `configs/point2a.yaml`.

9.1.2 Data Pipeline

In this step, the training data has been augmented with on the fly **random horizontal flipping** with probability $p = 0.5$.

9.1.3 Results Analysis

The training results are shown in Figure 4. The model achieved a final accuracy on the test set of approximately **33.2%**.

- **Loss and Overfitting:** The training appears more stable compared to the point1 exercise, showing the training and validation loss diverging only at the very end of the training process.
- **Confusion Matrix:** The Confusion Matrix (Figure 5) exhibits a little bit clearer diagonal, indicating that the model makes more correct predictions compared to the previous model. Classes like *Bedroom* and *Store* are now classified with accuracy comparable to some other classes. However, the model still struggles to differentiate between some classes like *Kitchen*, *Bedroom*, *Office*, and *InsideCity*.



Figure 4: Training metrics for CNN_2a. The vertical line marks the early stopping point.

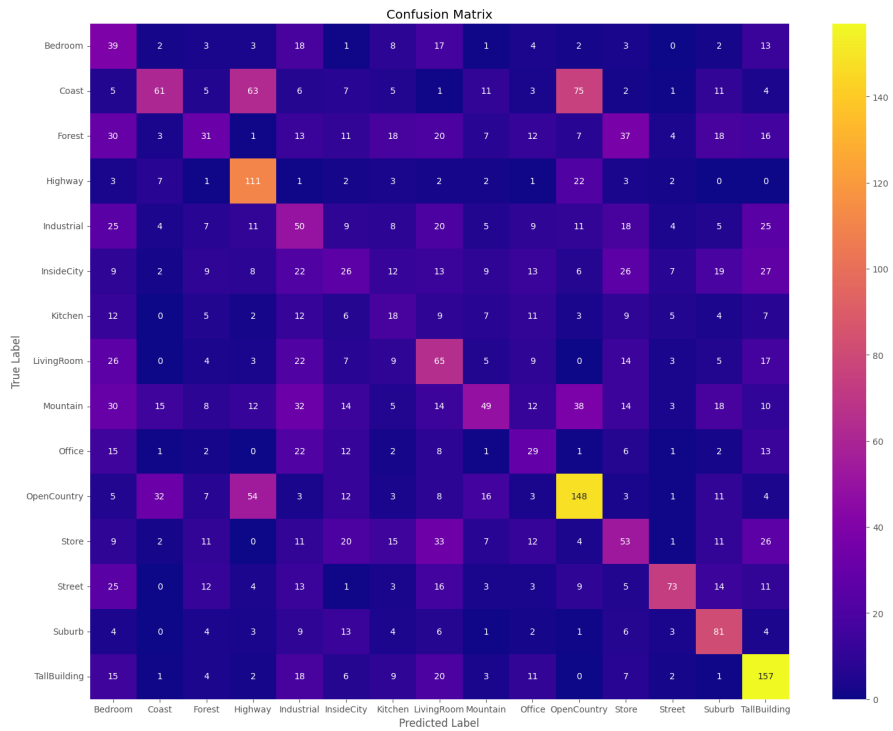


Figure 5: Confusion Matrix for CNN_2a on the test set.

9.2 Point 2b

This point can be solved using:

```
python solve.py --config_path='configs/point2b.yaml'
```

9.2.1 Network Architecture

The network architecture is defined in `configs/point2b.yaml`. It is similar to the previous Networks, with the addition of a Batch normalization layer before the ReLU activation functions in the convolutions Blocks. The specific structure is:

- **Block 1:** Conv2d (8 filters, 3×3) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2).
- **Block 2:** Conv2d (16 filters, 3×3) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2).
- **Block 3:** Conv2d (32 filters, 3×3) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU.
- **Classifier:** Flatten $\rightarrow$ Linear layer for the incoming dimension to 15, the output logits.

9.2.2 Data Pipeline

The data pipeline is the same as in the previous step, with an augmented training set with on the fly **random horizontal flipping** with probability $p = 0.5$.

9.2.3 Results Analysis

The training results are shown in Figure 6. The model achieved a remarkable final accuracy on the test set of approximately **50.2%**, which is significant for such a simple network architecture.

- **Loss and Overfitting:** The training loss appears stable and decreases consistently. However, the validation loss is unstable, which diverges immediately, remains around the same value, but with a high variance.

Applying normalization before the activation stabilizes the input distribution for each layer reducing *Internal Covariate Shift*. This makes each gradient step more effective and independent of the scale of previous layers, allowing for faster convergence without the need for a 'warm-up' phase.

- **Convergence Speed:** There is a clear difference with respect to the previous two plots: now the both losses decrease immediately without the initial "warm-up" phase seen in the earlier epochs of the previous models. The accuracy plot exhibits similar behavior, starting higher and improving faster.

- **Confusion Matrix:** The Confusion Matrix (Figure 7) exhibits a clear diagonal, indicating that the model makes a lot of correct predictions. The model still struggles to differentiate between similar classes like *Kitchen*, *Living room*, *Bedroom* and *Office*.

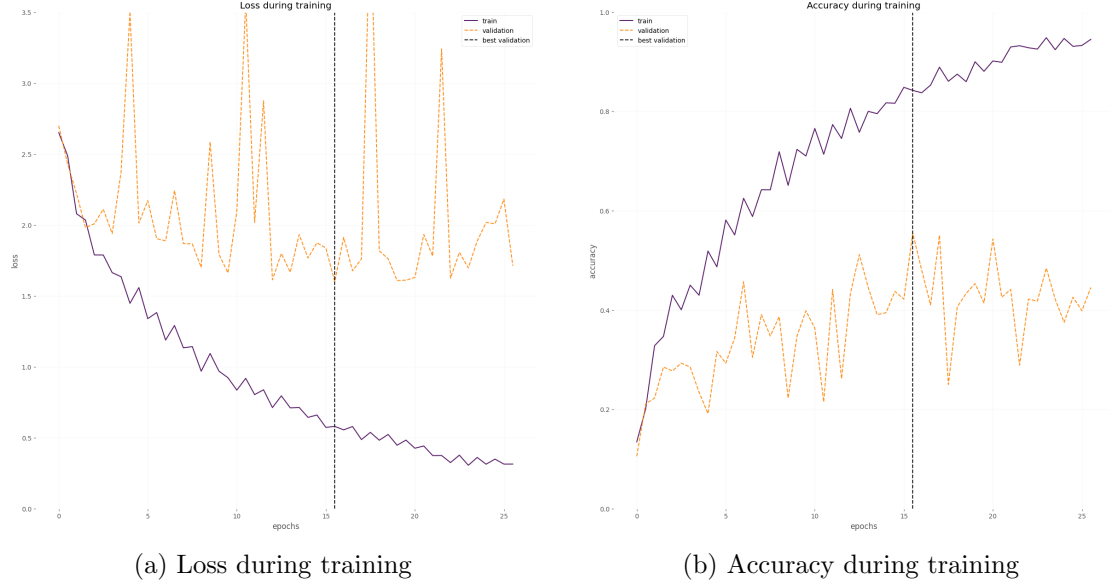


Figure 6: Training metrics for CNN_2b. The vertical line marks the early stopping point.

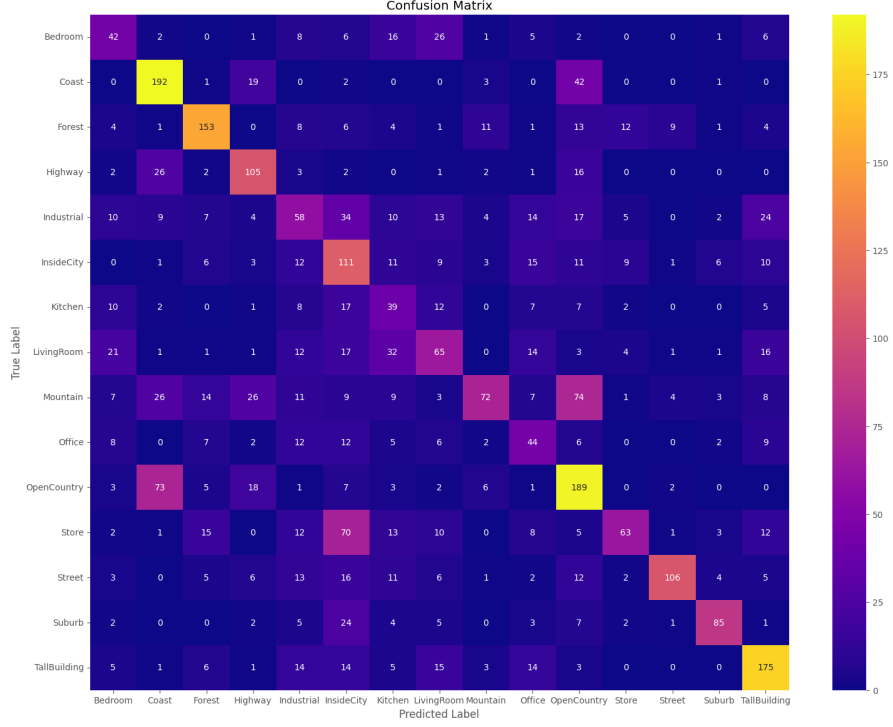


Figure 7: Confusion Matrix for CNN_2b on the test set.

9.3 Point 2c

This point can be solved using:

```
python solve.py --config_path='configs/point2c.yaml'
```

9.3.1 Network Architecture

The network architecture is defined in `configs/point2c.yaml` and has bigger blocks to allow the network to learn more complex features at each step. The specific structure is:

- **Block 1:** Conv2d (8 filters, 3×3 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Conv2d (8 filters, 3×3) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2).
- **Block 2:** Conv2d (16 filters, 5×5 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Conv2d (16 filters, 5×5) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2).
- **Block 3:** Conv2d (32 filters, 7×7) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU.
- **Classifier:** Flatten $\rightarrow$ Linear layer for the incoming dimension to 15, the output logits.

In this configuration, the model has approximately 52K parameters, which is fewer than the previous networks.

9.3.2 Data Pipeline

The data pipeline is the same as in the previous step, with an augmented training set with on the fly **random horizontal flipping** with probability $p = 0.5$.

9.3.3 Results Analysis

The training results are shown in Figure 8. The model achieved a final accuracy on the test set of approximately **53.4%**, slightly higher with respect to the previous networks even though it has fewer parameters.

- **Loss and Overfitting:** The training and validation losses appear stable and decrease consistently. A low patience was set, causing the training to stop early; otherwise, we would likely have obtained similar behavior as before if we had proceeded with training.
- **Confusion Matrix:** The Confusion Matrix (Figure 9) is very close to the previous one (Figure 7).



Figure 8: Training metrics for CNN_2c. The vertical line marks the early stopping point.

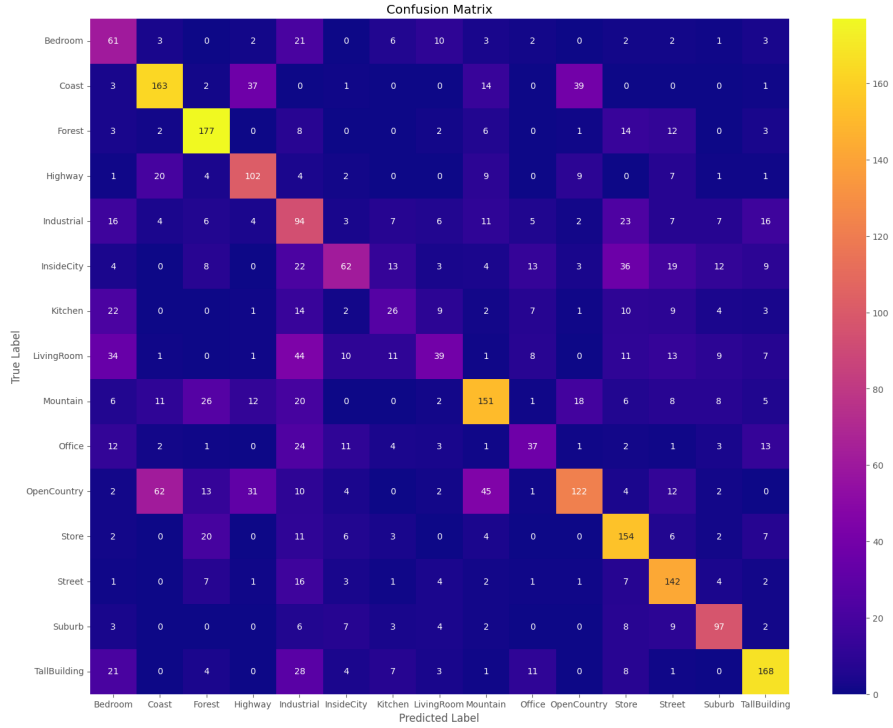


Figure 9: Confusion Matrix for CNN_2c on the test set.

9.4 Point 2d

This point can be solved using:

```
python solve.py --config_path='configs/point2d.yaml'
```

9.4.1 Network Architecture

The network architecture is defined in `configs/point2d.yaml` and is the same as in the previous step.

The weights were initialized using the `kaiming_normal_` method with 'fan_out' mode for convolutions and 'fan_in' mode for linear layers. An LLM was used to correctly apply this initialization in order to enhance network performance and training efficacy.

Since ReLU sets to zero all negative inputs values, the **Kaiming** initialization method initialize the weights with larger variance to compensate. This ensures the signal remains strong and doesn't vanish as it travels through the network.

- **fan_in** (used for Linear layers): It preserves the magnitude of the signal during the **forward pass**.

- **fan_out** (used for Convolutions): It preserves the magnitude of the gradients during the **backward pass**. This is often preferred for convolutional layers to ensure the network learns effectively during backpropagation.

The model was trained using the **AdamW** optimizer with a learning rate of 4.1×10^{-3} and a learning rate decay of 0.1.

AdamW is a variation of the popular Adam optimizer, which, differently from standard SGD, adapts the learning rate for each parameter individually based on gradient statistics. The “**W**” stands for **Weight Decay**. In AdamW, weight decay is applied directly to the weights rather than being added to the loss function like L2 regularization.

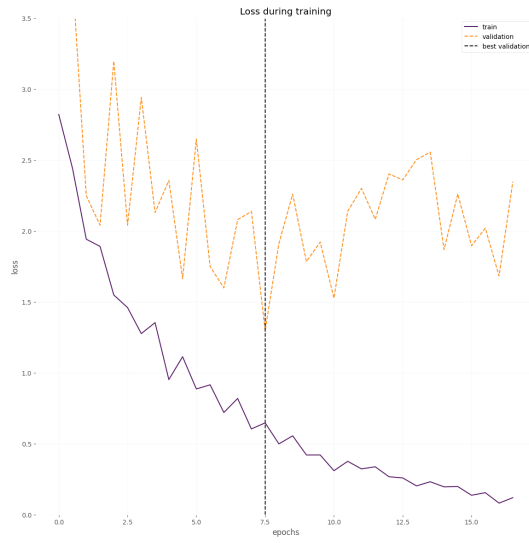
9.4.2 Data Pipeline

The data pipeline is the similar as in the previous step, with an augmented training set with on the fly **random horizontal flipping** with probability $p = 0.5$ and with a **Normalization** of the data, so that each pixel value falls between 0 and 1.

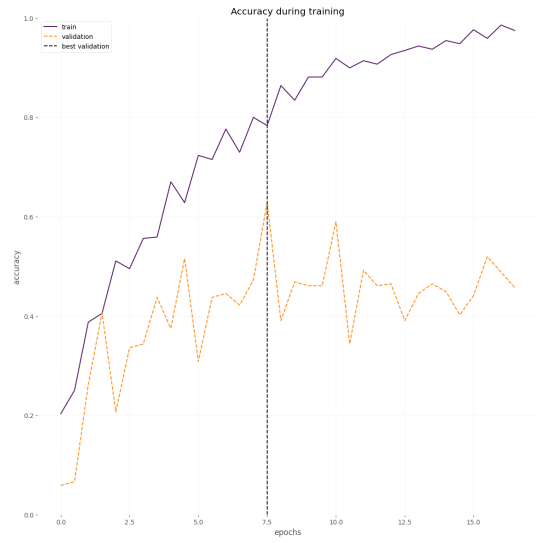
9.4.3 Results Analysis

The training results are shown in Figure 10. The model achieved a final accuracy on the test set of approximately **57%**, slightly higher with respect to the previous network.

- **Loss and Overfitting:** The training and validation losses appear similar to those in Point 2b (Figure 6), but with **lower variance** in the validation loss after overfitting begins.
- **Confusion Matrix:** The Confusion Matrix (Figure 11) exhibits a clear diagonal, indicating a high number of correct predictions. Similar classes like *Kitchen*, *Living Room*, *Bedroom*, and *OpenCountry*, *Coast* are the ones most frequently misclassified. However, distinct categories are generally well separated.



(a) Loss during training



(b) Accuracy during training

Figure 10: Training metrics for CNN_2d. The vertical line marks the early stopping point.

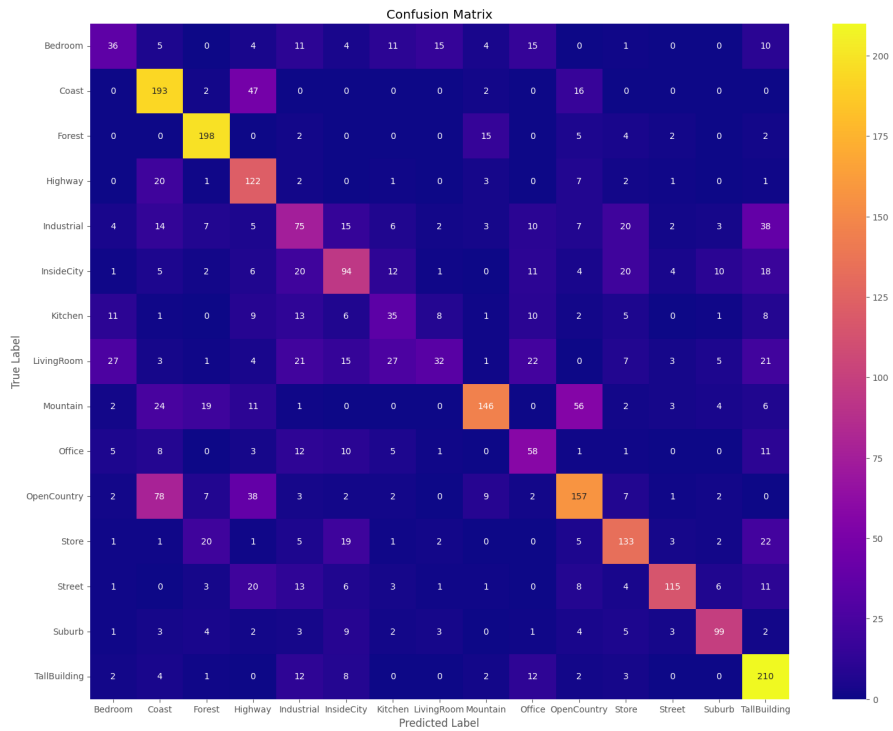


Figure 11: Confusion Matrix for CNN_2d on the test set.

9.5 Point 2e

This point can be solved using:

```
python solve.py --config_path='configs/point2e.yaml'
```

9.5.1 Network Architecture

The network architecture is defined in `configs/point2e.yaml`. It is the same as the previous configuration but with the addition of **Dropout** layers. The specific structure is:

- **Block 1:** Conv2d (8 filters, 3×3 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Conv2d (8 filters, 3×3) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2) $\rightarrow$ Dropout ($p = 0.5$).
- **Block 2:** Conv2d (16 filters, 5×5 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Conv2d (16 filters, 5×5) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2) $\rightarrow$ Dropout ($p = 0.5$).
- **Block 3:** Conv2d (32 filters, 7×7) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Dropout ($p = 0.2$).
- **Classifier:** Flatten $\rightarrow$ Linear layer mapping the incoming features to 15 output logits.

By randomly zeroing out elements of the input tensor with probability p during training, Dropout forces the network to learn more robust features that does not depend on specific neurons, effectively acting as a regularization technique.

9.5.2 Data Pipeline

The data pipeline is the same as in the previous step, with an augmented training set with on the fly **random horizontal flipping** with probability $p = 0.5$ and with input normalization.

9.5.3 Results Analysis

The training results are shown in Figure 12. The model achieved a final accuracy on the test set of approximately **59.4%**, which is slightly higher with respect to the previous networks.

- **Loss and Overfitting:** The losses appear decreasing consistently on both the training and validation sets. This demonstrates the efficacy of dropout in reducing overfitting. It can be seen, also, that the two losses are close at the end of the training process.

- **Confusion Matrix:** The Confusion Matrix (Figure 13) is close to the previous one (Figure 11), though the misclassifications appear slightly more scattered across the classes.

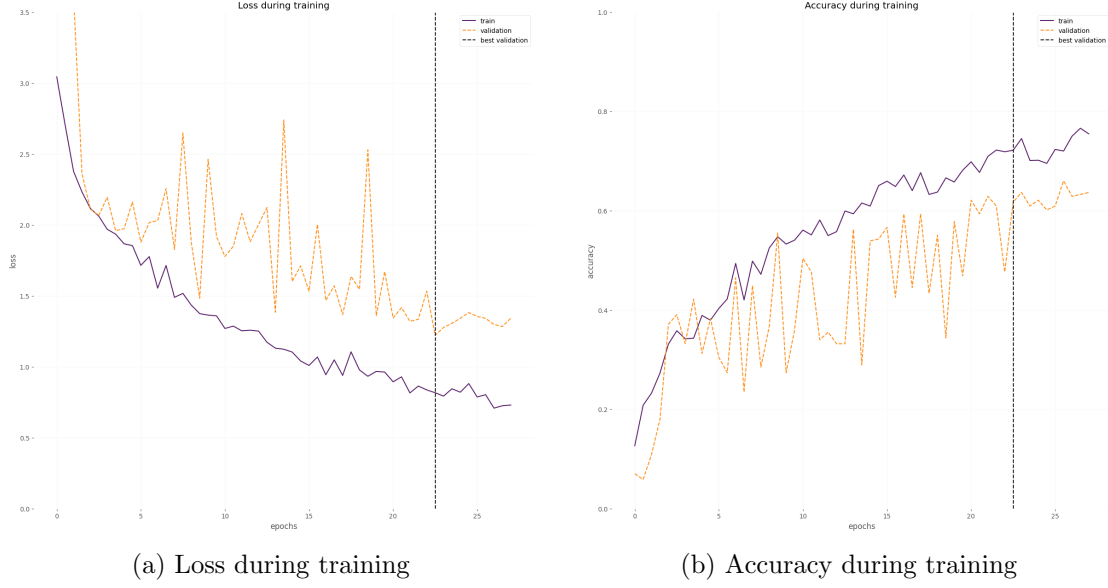


Figure 12: Training metrics for CNN_2e. The vertical line marks the early stopping point.

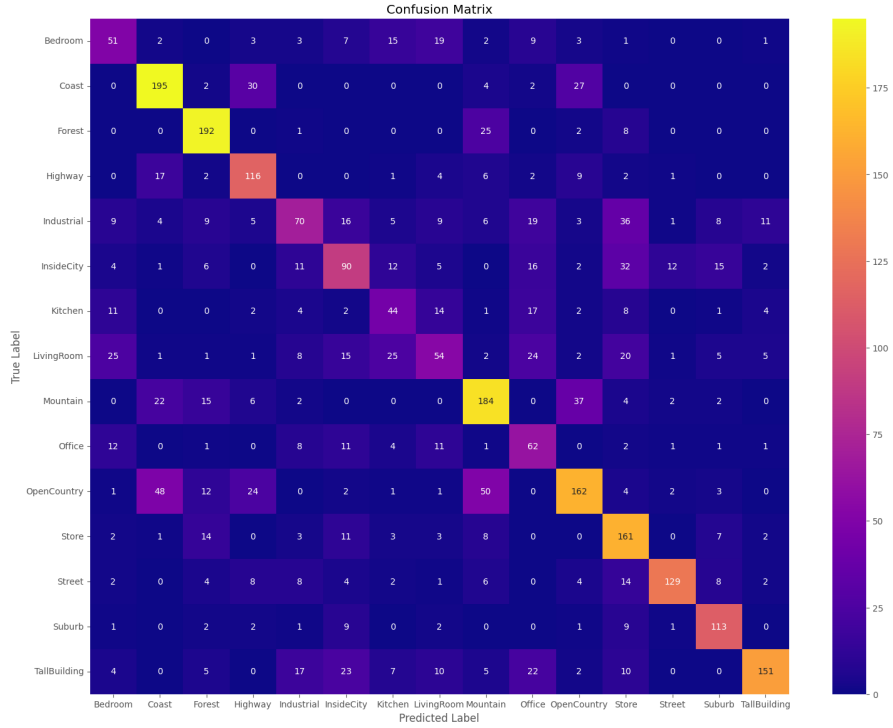


Figure 13: Confusion Matrix for CNN_2e on the test set.

9.6 Point 2f

This point can be solved using:

```
python point2f.py --config_path='configs/point2f.yaml'
```

9.6.1 Network Architecture

The network architecture is defined an **ensemble of 9 networks**, in which each one has the same configuration as before. It was, however, redefined in `configs/point2f.yaml`.

9.6.2 Data Pipeline

The data pipeline is the same as in the previous step, with an augmented training set with on the fly **random horizontal flipping** with probability $p = 0.5$ and with input normalization.

9.6.3 Results Analysis

The training results are shown in Figure 14. The model achieved a final accuracy on the test set of approximately **63.1%**, higher with respect to the previous networks.

- **Loss and Overfitting:** Single loss plot can be found in `results/point2f/` and they are basically all similar to the previous step, since nothing has changed in the setup.
- **Confusion Matrix:** The Confusion Matrix (Figure 14) is close to the previous one (Figure 13), with a little less sparse wrong predictions.

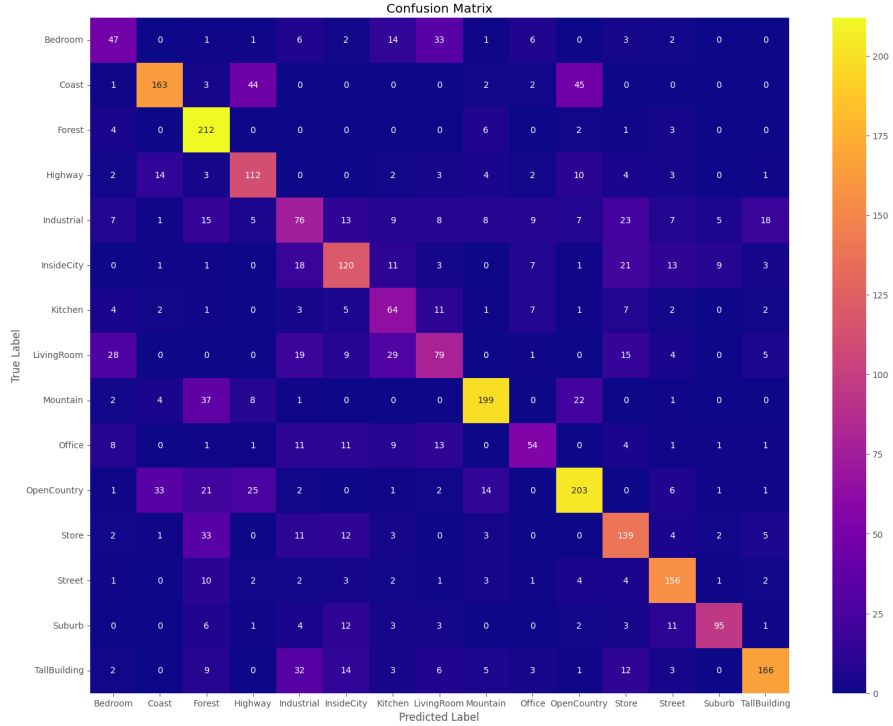


Figure 14: Confusion Matrix for CNN_2f on the test set.

10 Point 3

10.1 Point 3a

This point can be solved using:

```
python point3a.py --config_path='configs/point3a.yaml'
```

This point requires loading a pretrained **AlexNet**² and performing fine-tuning by substituting the last layer of the network.

²Krizhevsky et al., 2012: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

10.1.1 Network Architecture

The architecture is the standard AlexNet neural network, with the substitution of the final linear layer. The new layer takes 4096 input features (the one computed by the AlexNet layers) and maps them to 15 output features, which corresponds to the number of classes in our dataset.

The full network has **57 million** parameters, making it approximately 10^3 times bigger than the previous networks. However, we train only the last layer, which has around 61K parameters.

10.1.2 Data Pipeline

The data pipeline follows the procedure described in the PyTorch documentation³. It expects images to be rescaled to 256×256 , followed by a random crop to 224×224 , and finally normalized with specific mean and standard deviation values.

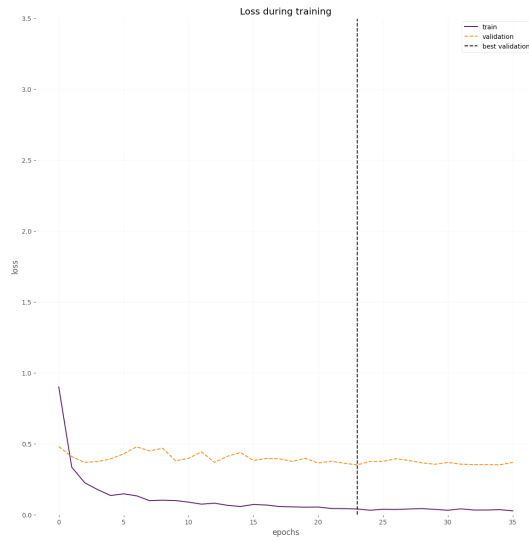
The test and validation pipelines perform only a rescaling to 224×224 and normalization.

10.1.3 Results Analysis

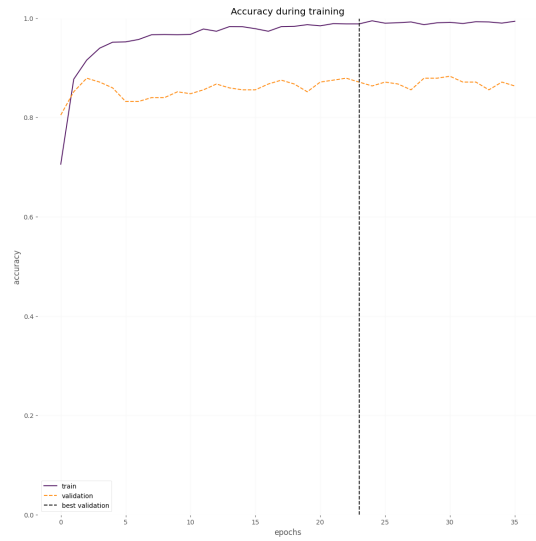
The training results are shown in Figure 15. The model achieved a final accuracy on the test set of approximately **85.2%**, which is significantly higher than the previous networks.

- **Loss and Overfitting:** The loss decreases very fast and reaches good performance almost immediately. After the initial drop, the training loss continues to decrease very slowly, while the validation loss stabilizes.
- **Confusion Matrix:** The Confusion Matrix (Figure 16) exhibits a very clear diagonal, indicating a high number of correct predictions. Similar classes like *Kitchen*, *Living Room*, and *Bedroom* are still the ones most frequently misclassified, but the errors are much lower than before.

³https://pytorch.org/hub/pytorch_vision_alexnet/



(a) Loss during training



(b) Accuracy during training

Figure 15: Training metrics for AlexNet finetuning. The vertical line marks the early stopping point.

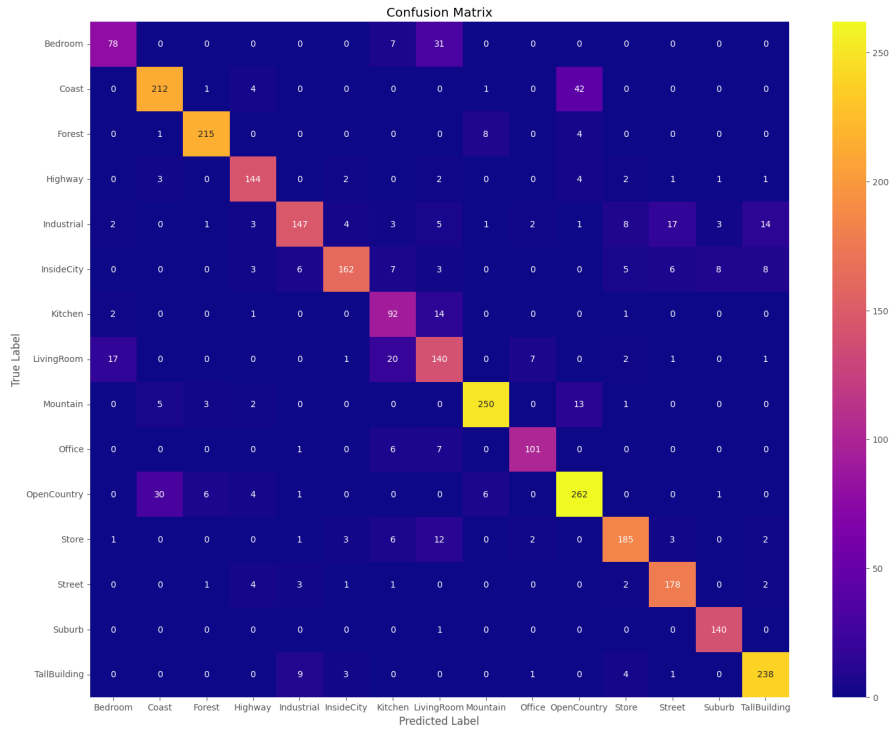


Figure 16: Confusion Matrix for AlexNet finetuning on the test set.

10.2 Point 3b

This point can be solved using:

```
python pointSVM.py --config_path='configs/point3b.yaml'
```

This point requires loading a pretrained **AlexNet** and using it as a feature extractor. The output of the penultimate layer is used to train a Multiclass Support Vector Machine (SVM). In this point, we will use a **linear kernel**.

10.2.1 Network Architecture

The architecture is the standard AlexNet neural network, with the substitution of final linear layer with an Identity layer. Consequently, the network outputs the computed feature vector directly.

10.2.2 Multiclass SVM

Since SVMs are inherently binary classifiers, we use specific strategies to extend them to the multiclass problem (15 classes):

- **OVO (One-vs-One):** This is the default method for the scikit learn SVC class, and breaks the multiclass problem into multiple binary problems. For N classes, it trains $\frac{N(N-1)}{2}$ binary classifiers, where each classifier learns to distinguish between a specific pair of classes. During inference, a voting scheme is applied: every classifier casts a vote, and the class with the most votes is the final prediction.
- **DAG (Directed Acyclic Graph):** This method, coded from scratch using the old course notes, uses the same number of binary classifiers as OVO but optimizes the prediction phase. It organizes the classifiers into a Directed Acyclic Graph structure. During inference, we traverse the graph starting from the root. At each node, a binary classifier compares two classes; the "losing" class is eliminated, and we move to the next node. This process repeats until only one class remains. This makes inference significantly faster than the standard voting approach.

The DAG SVM code can be found in `models/SVMs.py`

10.2.3 Data Pipeline

The data preprocessing pipeline is the same as in the previous step. A new dataset is computed: it is no more composed of the images themselves, but rather of the feature vectors of 4096 elements extracted by AlexNet.

10.2.4 Results Analysis

The models achieved a final accuracy on the test set of approximately **85.3%** for both the DAG and OVO SVMs, which is essentially equal to the result from the fine-tuning step (Point 3a).

- **Confusion Matrix:** The Confusion Matrices for both methods (Figure 17) are very similar and exhibit a clear diagonal, indicating a high number of correct predictions. They are also remarkably similar to the AlexNet fine-tuning matrix. This is reasonable because both approaches perform a linear operation on the same deep features: the fine-tuning trains a Linear Layer (Softmax), while this step trains Linear SVMs.

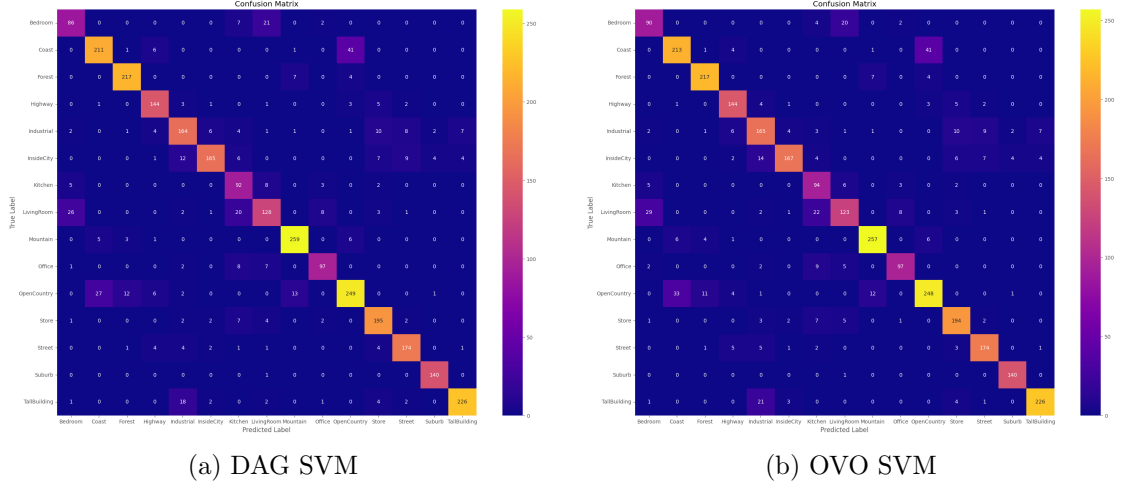


Figure 17: Confusion Matrix for SVM prediction using AlexNet features on the test set.

11 Point 4

This point can be solved using:

```
python solve.py --config_path='configs/point4b.yaml'
```

11.0.1 Network Architecture

The network architecture is defined in `configs/point4a.yaml`. It is the same network as point2e.

11.0.2 Data Pipeline

The data pipeline has an augmented training set with on the fly **random horizontal flipping**, **random rotation** with bilinear interpolation and **random cropping**, with input normalization.

11.0.3 Results Analysis

The training results are shown in Figure 18. The model achieved a final accuracy on the test set of approximately **59.3%**, which is basically the same as point2e. The augmentation seems not to have a big impact.



Figure 18: Training metrics for CNN_4b. The vertical line marks the early stopping point.

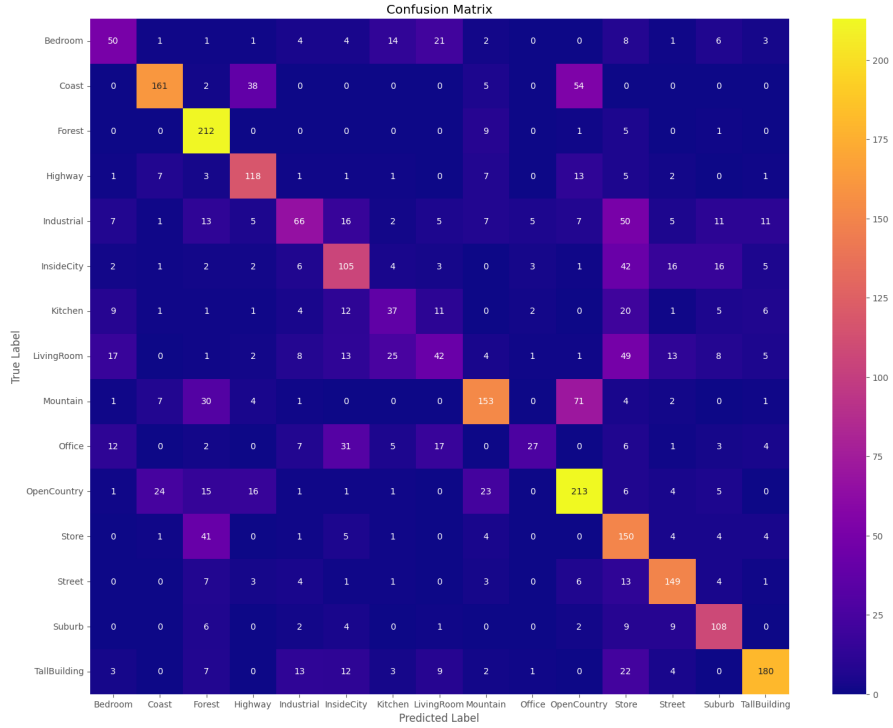


Figure 19: Confusion Matrix for CNN_4b on the test set.

12 Point 5

This point can be solved using:

```
python solve.py --config_path='configs/point5a.yaml'
```

12.1 Network Architecture

The network architecture is defined in `configs/point5a.yaml`. It is a bigger network compared to the previous one, with more blocks and more filters. The specific structure is:

- **Block 1:** Conv2d (8 filters, 3×3 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Conv2d (8 filters, 3×3 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2) $\rightarrow$ Dropout ($p = 0.3$).
- **Block 2:** Conv2d (16 filters, 5×5 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Conv2d (16 filters, 5×5 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2) $\rightarrow$ Dropout ($p = 0.3$).

- **Block 3:** Conv2d (32 filters, 5×5 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Conv2d (32 filters, 5×5 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2) $\rightarrow$ Dropout ($p = 0.3$).
- **Block 4:** Conv2d (64 filters, 5×5 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ Conv2d (64 filters, 5×5 , padding=(1, 1)) $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2) $\rightarrow$ Dropout ($p = 0.2$).
- **Classifier:** Flatten $\rightarrow$ Linear layer mapping the incoming features to 15 output logits.

This network has approximately 85 K parameters.

12.2 Data Pipeline

The data pipeline has an augmented training set with on the fly **random horizontal flipping**, **random rotation** with bilinear interpolation and **random cropping**, with input normalization.

12.3 Results Analysis

The training results are shown in Figure 12. The model achieved a final accuracy on the test set of approximately **64.9%**, which is slightly higher with respect to the previous networks trained from scratch.

- **Loss and Overfitting:** The losses appear decreasing consistently on both the training and validation sets.
- **Confusion Matrix:** The Confusion Matrix (Figure 21) is close to the point 2e confusion matrix (Figure 13).



Figure 20: Training metrics for CNN_4a. The vertical line marks the early stopping point.

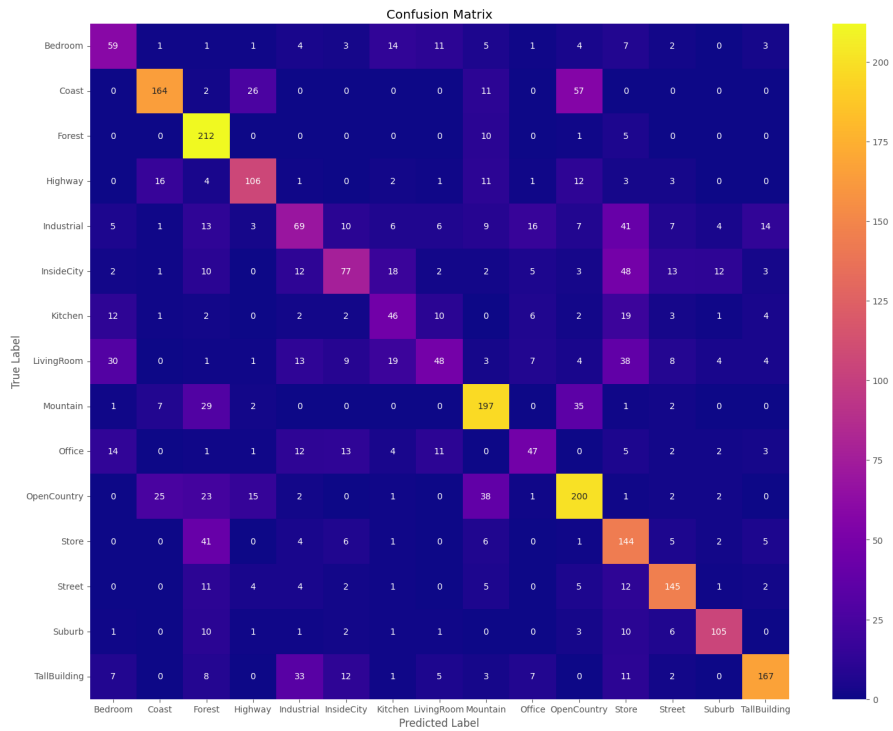


Figure 21: Confusion Matrix for CNN_4a on the test set.

13 Point 6

This point can be solved using:

```
python pointSVM.py --config_path='configs/point6.yaml'
python pointSVM.py --config_path='configs/point6ovo.yaml'
```

This point requires loading a pretrained **AlexNet** and using it as a feature extractor. The output of the penultimate layer is used to train a Multiclass Support Vector Machine (SVM). In this point, we will use a **rbf (Gaussian) kernel**.

13.1 Data Pipeline

The data data pipeline is the same as the point 3b.

13.2 Results Analysis

The models achieved a final accuracy on the test set of approximately **86.2%** for the DAG method and **86.3%** for the OVO method, which is 1% higher than the results with linear kernel Point 3b.

- **Confusion Matrix:** The Confusion Matrices for both methods (Figure 22) are very similar and exhibit a clear diagonal, indicating a high number of correct predictions. They are also similar to the confusion matrix of other AlexNet points, since one percent of higher accuracy is not so visible in the confusion matrix.

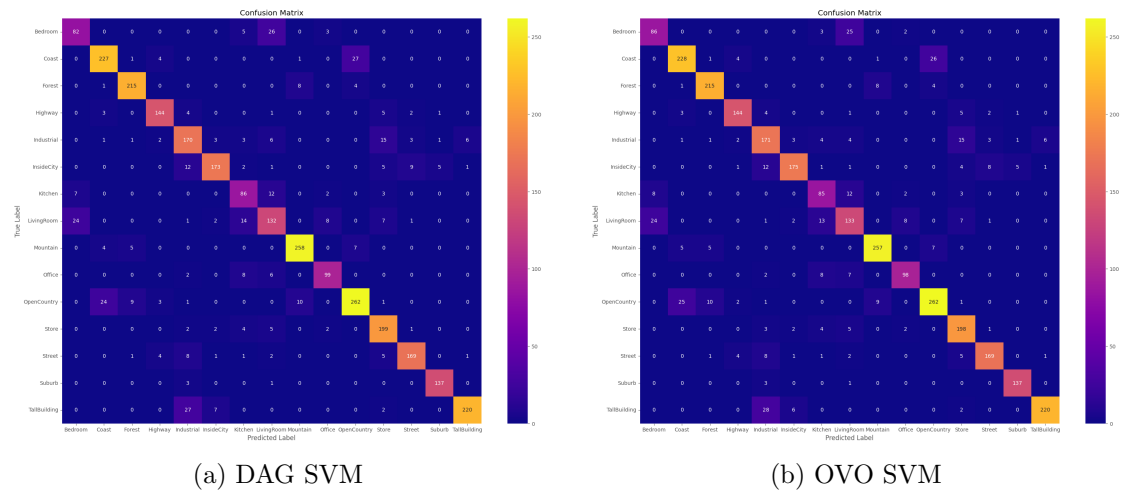


Figure 22: Confusion Matrix for SVM prediction using AlexNet features on the test set.

14 Point 7

This point can be solved using:

```
python pointSVM.py --config_path='configs/point7.yaml'
python pointSVM.py --config_path='configs/point7rbf.yaml'
```

This point requires loading a pretrained **AlexNet** and using it as a feature extractor. The output of the penultimate layer is used to train a Multiclass Support Vector Machine (SVM) implemented in:

`models/SVMs.py`

In this file there is the implementation from scratch of the **Error Correcting Output Codes (ECOC)** approach to handle the multiclass classification problem. An LLM was used to better understand the paper and to improve and correct already written code.

14.1 Error Correcting Output Code (ECOC)

Unlike the One-vs-One (OVO) or Directed Acyclic Graph (DAG) approaches, which break the problem into pairs of classes, the ECOC approach assigns a unique binary code (codeword) of length L to each class. The training process involves L independent binary classifiers:

- **Codebook Generation:** A codebook matrix is generated where each row represents a class and each column represents a binary classifier as shown in the Figure 23. The generation is performed using the Randomized Hill Climbing method described in the paper at point 2.3.3.⁴.
- **Training:** Each of the L binary SVMs is trained on the **entire dataset**. For a specific SVM (column j), the dataset is split into two new sub classes: samples belonging to classes with a '0' in the j -th column of the codebook vs. samples with a '1'.
- **Inference (Error Correction):** During prediction, all L classifiers output a bit, forming a predicted binary word. The final class is determined by calculating the **Hamming distance** between this predicted word and the codewords in the codebook. The class with the closest codeword is selected. This provides robustness: even if a few individual classifiers make mistakes, the correct class can still be recovered if the code distance is sufficiently large.

⁴Dietterich and Bakiri, 1994: <https://arxiv.org/pdf/cs/9501101>

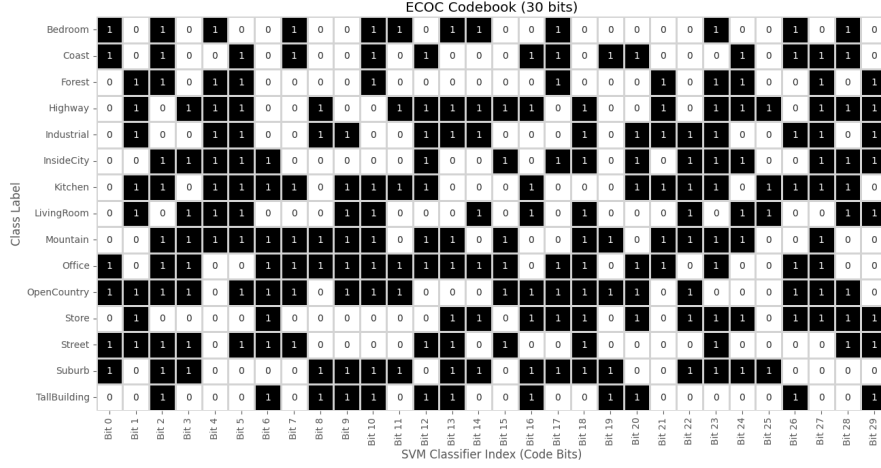


Figure 23: Plot of a codebook with $L = 30$ bits

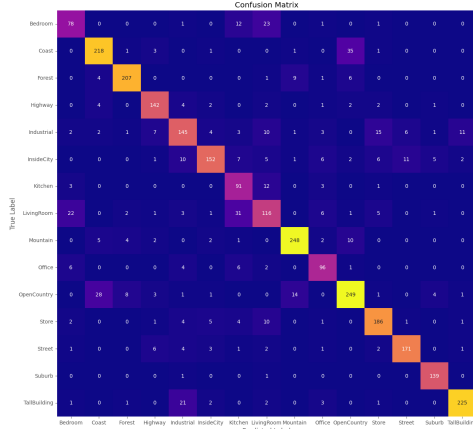
14.2 Data Pipeline

The data pipeline is the same as in Point 3b (AlexNet feature extraction).

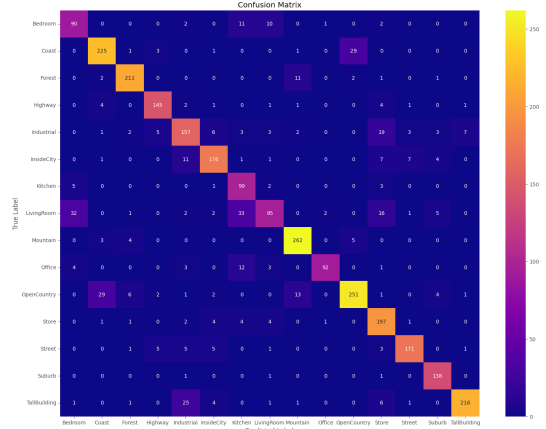
14.3 Results Analysis

The models achieved a final accuracy on the test set of approximately **82.5%** for the Linear kernel and **84.6%** for the RBF kernel. These results are slightly lower than those obtained with the DAG or OVO methods.

- **Confusion Matrix:** The Confusion Matrices for both kernels (Figure 24) are very similar and exhibit a clear diagonal, indicating a high number of correct predictions. They are visually similar with the results of the other points that use AlexNet, as the small difference in percentage accuracy is not easily discernible in the confusion matrix visualization.



(a) Linear kernel



(b) RBF kernel

Figure 24: Confusion Matrix for SVM ECOC prediction using AlexNet features on the test set.

15 Vision Transformer

A Vision Transformer (ViT) architecture was also tested. The implementation code can be found in:

`models/ViT.py`

It can be trained and tested using:

`python pointViT.py --config-path='configs/vit.yaml'`

15.1 Network Architecture

The network is a classic Vision Transformer with three transformer blocks, each with 4 attention heads. The input image is divided into **patches** of dimension 4×4 . The embedding dimension for each patch is set to 16. Since the input is a grayscale image, the flattened patch size is $4 \times 4 \times 1 = 16$. This means the linear projection step maps the patch directly to the embedding space without expanding or compressing the dimensionality initially.

The total number of parameters is approximately 10K, which is significantly lower than the previously tested Convolutional networks. Interestingly, increasing the number of parameters did not consistently lead to improved performance and other investigation are needed.

The weights were initialized using default methods and the model was trained using the **AdamW** optimizer.

15.2 Results Analysis

The model achieved a final accuracy on the test set of approximately **38.86%**. While lower than the best CNNs, this result is remarkable considering the extremely small size of the network (10K parameters) and the lack of inductive biases (like convolutions).

- **Loss and Overfitting:** In the training plot (Figure 25), we can see that the training loss and validation loss remain close to each other throughout the process, indicating **no overfitting**. It is worth noting that this model takes longer to converge compared to CNNs, showing a slow but steady improvement over time.
- **Confusion Matrix:** The Confusion Matrix (Figure 26) exhibits a slight diagonal, indicating that the model is making correct predictions. Interestingly, classes like *Bedroom* and *Kitchen* are frequently misclassified, while other classes are predicted with higher accuracy.

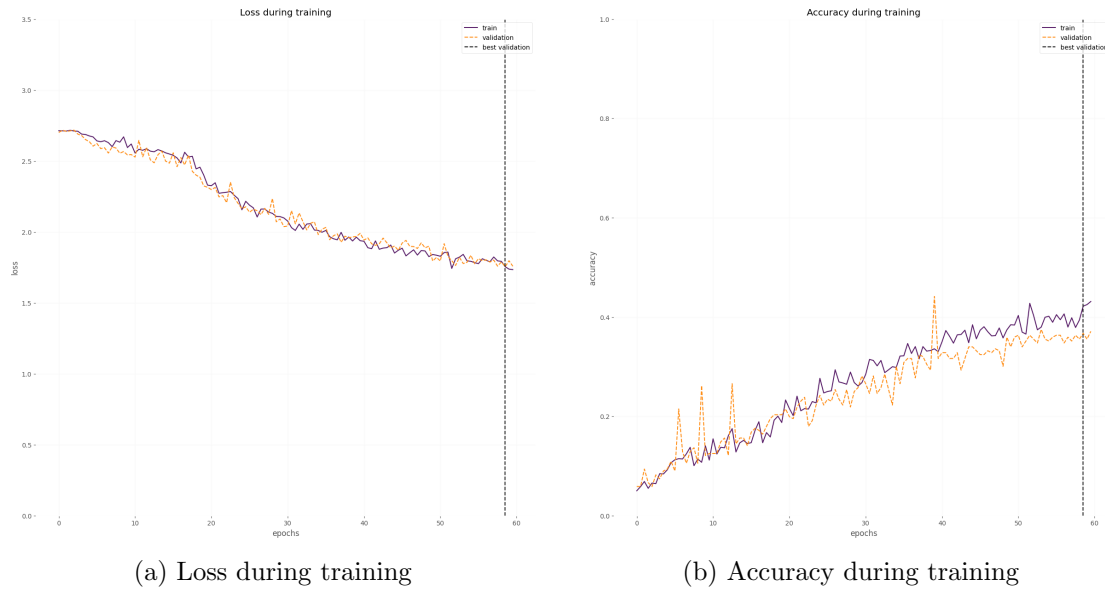


Figure 25: Training metrics for ViT. The vertical line marks the early stopping point.

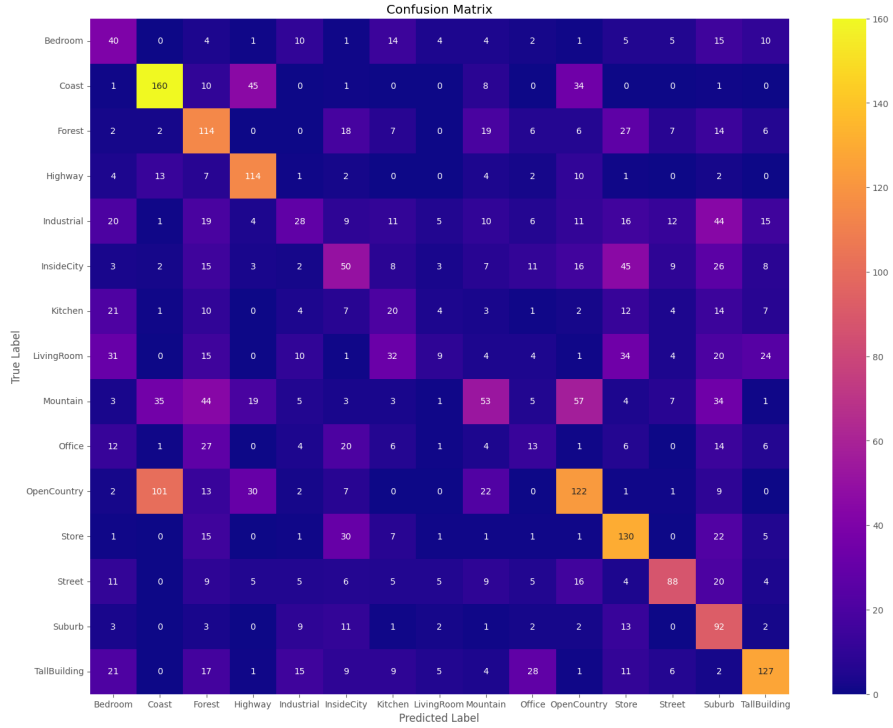


Figure 26: Confusion Matrix for ViT model on the test set.

16 Conclusion

In this project, we compared training simple CNNs from scratch against using pre-trained models for image classification.

Our initial custom CNNs struggled with overfitting, achieving only $\approx 25\%$ accuracy. We incrementally improved this by adding regularization, stability and ensembling.

However, the best model were the one that use **Transfer Learning**. Using a pre-trained like AlexNet as a feature extractor immediately jumped the accuracy to over **85%**, even though it means use a very big network. We confirmed this result using both a simple linear layer and various types of multiclass SVMs, proving that learned features from large datasets are far more powerful than complex architectures trained on small dataset.